

1 Companion 1: Arithmetic Calibration Companion (v19.0.0 complete repair)

1.1 C1.1. Scope and non-theorem status

This companion document provides operational calibration material for the arithmetic interface appendices of the AK framework. It is a companion document, not a theorem-bearing appendix. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this companion is:

Arithmetic calibration examples are not arithmetic theorems.

The purpose of this companion is to provide templates, calibration records, example schemas, failure patterns, and non-claim examples for arithmetic interfaces. It supports the use of:

Appendix MS-A, Appendix MS-B, Appendix MS-C, Appendix MS-F.

It does not prove the Birch–Swinnerton-Dyer conjecture, the ABC conjecture, the Riemann hypothesis, Iwasawa main conjectures, modularity theorems, congruence theorems, mirror symmetry, or Fukaya-categorical theorems.

No calibration record, example file, arithmetic template, tower record, bridge sketch, or operational manifest may replace:

$$\begin{aligned} &B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \\ &(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}). \end{aligned}$$

Remark 1.1 (Companion status). This document is operational and illustrative. It may contain templates and examples that help instantiate the arithmetic interface protocol. It does not create Core theorem status, interface theorem status, bridge theorem status, or arithmetic theorem status.

Remark 1.2 (Relation to Part IV). Part IV defines the problem-interface discipline. This companion supplies calibration material for using that discipline in arithmetic settings. When in doubt, the formal definitions and exclusions in Appendices MS-A–MS-F take precedence over examples in this companion.

Remark 1.3 (Companion boundary). A companion may show how a record should be shaped. It may not promote the record from example status to theorem status. Promotion requires the relevant verifier-side or bridge-theorem evidence.

1.2 C1.1bis. v19 companion hardening and verifier-side boundary

Definition 1.4 (Companion-sovereignty boundary). The companion-sovereignty boundary is the rule that an arithmetic calibration object, example schema, mock record, bridge sketch, template, or non-claim example remains operational companion material unless it is converted into a declared verifier-side or bridge-theorem record and passes the applicable protocol. This boundary applies even when the example is internally well formed, checked by a human, stored in a platform, or copied into an audit manifest.

Definition 1.5 (Calibration-to-verifier conversion record). A calibration-to-verifier conversion record is required whenever a calibration object is consumed as more than an example. It declares the source calibration object, target verifier-side schema, source and target scopes, required hypotheses, artifact identities, claim-use preflight result, resulting status *pass/reject/undefined*, and typed failure route. Without this record, the calibration object remains a companion template or non-claim example.

Definition 1.6 (Companion scope-exclusion policy). A template field written as `not_applicable` or `not_used` maps to `not_invoked` only when the record contains a declared scope-exclusion policy explaining why the clause is outside the run scope. Absent such a policy, theorem-facing consumption of the missing field is undefined, not pass.

Proposition 1.7 (Checked calibration is non-verdict). *A checked calibration does not determine Core pass, interface theorem status, bridge theorem status, or arithmetic theorem status.*

Proof. Checked calibration verifies companion-level well-formedness only. Core and bridge verdicts consume verifier-side records, artifacts, ledgers, diagnostics, claim-use preflight, and proofs at their declared types. A companion-level shape check supplies none of those obligations by itself. $\square$

Proposition 1.8 (Companion examples cannot discharge non-scalar obligations). *No arithmetic companion example discharges representative existence, eligibility, terminal tameness, artifact identity, claim-use permission, or tower-comparison existence.*

Proof. These are non-scalar verifier-side obligations under the v19 Core guard-rails. An example can illustrate the field in which evidence would be recorded, but it is not the evidence itself. $\square$

Remark 1.9 (Companion ledger-token discipline). In companion templates, ledger names such as `delta_arith`, `delta_cong`, `delta_Iw`, `delta_tower`, and `delta_bridge` are local illustration tokens. They enter a metric budget only through Appendix M registration, support records, scalarization, and a bottleneck discrepancy bound.

1.3 C1.2. Calibration object taxonomy

Definition 1.10 (Arithmetic calibration object). An *arithmetic calibration object* is an illustrative or operational object used to test, configure, or document an arithmetic interface workflow. Examples include:

- (i) a sample arithmetic source datum;
- (ii) a sample cyclotomic tower;
- (iii) a sample Iwasawa module profile;
- (iv) a sample congruence defect record;
- (v) a sample realization template;
- (vi) a sample repaired evaluation record;
- (vii) a sample admissible representative record;
- (viii) a sample tower comparison artifact record;
- (ix) a sample ledger entry;
- (x) a sample Type IV proxy record;
- (xi) a sample monitored diagnostic record;
- (xii) a sample manifest entry;
- (xiii) a failure-case record.

Definition 1.11 (Calibration status). A calibration object has one of the following statuses:

- (i) example;
- (ii) template;
- (iii) mock record;
- (iv) calibration candidate;
- (v) checked calibration;
- (vi) failed calibration;
- (vii) non-claim example;
- (viii) deprecated example;
- (ix) superseded example.

Definition 1.12 (Checked calibration). A *checked calibration* is a calibration object whose fields are internally well-formed for the declared companion-level purpose. It is not a Core certificate unless separately submitted to and accepted by the Core verifier.

Remark 1.13 (Calibration is not certification). A calibration object may help test whether an interface record is well-formed. It does not certify the corresponding arithmetic claim.

1.4 C1.3. Arithmetic source data template

An arithmetic interface should begin by declaring its source data.

Definition 1.14 (Arithmetic source data calibration template). An *arithmetic source data calibration template* has the following fields:

- (i) source identifier;
- (ii) arithmetic family;
- (iii) base field or base scheme;
- (iv) coefficient field k ;
- (v) arithmetic objects;
- (vi) local conditions, if any;
- (vii) tower structure, if any;
- (viii) congruence structure, if any;
- (ix) realization purpose;
- (x) intended Core-readable target, if any;
- (xi) non-claim declaration.

A suggested schematic record is:

```

arithmetic_source:
  id: "arith-src-001"
  family: "elliptic_curve / abelian_variety / galois_representation / selmer_complex / iwasawa_module / congruence"
  base: "declared global field or arithmetic base"
  coefficient_field: "k"
  objects:
    - "object_1"
    - "object_2"
  local_conditions:
    status: "declared / not_used / unavailable"
    references: []
  tower:
    status: "none / cyclotomic / level / congruence / other"
    direction: "finite_to_limit / local_to_global / scale_refinement"
  congruence:
    status: "none / declared / advisory"
  realization_purpose: "PH1 / Ext-edge / tower diagnostic / ledger comparison / advisory"
  core_target:
    status: "none / FiltCh(k) / Perscons_k / Eval_{i,W,tau} / C_{tau,W}F / phi_{i,W,tau}"
  non_claim:
    - "This source declaration does not prove an arithmetic theorem."

```

Remark 1.15 (Purpose must be explicit). The same arithmetic object may support different interface purposes. For example, a Selmer-related object may be used for an advisory obstruction profile, a persistence realization, a tower diagnostic candidate, or a bridge candidate. The intended purpose must be declared before calibration can be evaluated.

1.5 C1.4. Cyclotomic tower calibration

Definition 1.16 (Cyclotomic tower calibration record). A *cyclotomic tower calibration record* documents how a cyclotomic or p -power tower is represented for Core-readable analysis. It must record:

- (i) tower layers;
- (ii) transition maps;
- (iii) finite-stage arithmetic data;
- (iv) limiting object policy;
- (v) realization route;
- (vi) repaired evaluation policy;
- (vii) tower comparison artifact target;
- (viii) monitored degrees;
- (ix) defect and ledger treatment;
- (x) non-claim declarations.

A suggested schematic record is:

```

cyclotomic_tower_calibration:
  id: "cyc-cal-001"
  prime: "p"
  layers:
    indexing: "n >= 0"
    layer_objects: ["A_0", "A_1", "..."]
    transition_maps: "declared"
  limiting_policy:
    object: "A_infty"
    status: "declared / conjectural / not_available"
  realization:
    functor_or_procedure: "R_cyc"
    target: "FiltCh(k) / Perscons_k"
    constructibility_check: "pass / reject / undefined"
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F_n) = T_bar_tau(W_W P_i(F_n))"
    monitored_degrees: ["i=0", "i=1"]
    windows: []
    threshold_policy: "declared"
  tower_artifact:
    comparison_artifact: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)"
    artifact_status: "candidate / declared / verified / rejected / undefined"
  ledger:
    delta_cyc: "value_or_bound"
    aggregation: "declared ledger semantics"
  non_claim:
    - "This calibration does not prove a cyclotomic control theorem."
    - "This calibration does not prove Type IV cleanliness."

```

Remark 1.17 (Cyclotomic tower versus Type IV tower). A cyclotomic tower is not automatically a Type IV diagnostic tower. It becomes diagnostically meaningful only after a Core-readable realization, repaired evaluation policy, and declared tower comparison artifact

$$\phi_{i,W,\tau}$$

are supplied.

1.6 C1.5. Iwasawa module calibration

Definition 1.18 (Iwasawa module calibration record). An *Iwasawa module calibration record* documents how an Iwasawa module or Iwasawa-layer object is represented in the AK interface. It must record:

- (i) Iwasawa algebra or tower;
- (ii) module or complex;
- (iii) arithmetic μ -invariant convention, if used;
- (iv) one-parameter filtration extraction;
- (v) Core-readable target;

- (vi) repaired evaluation policy;
- (vii) distinction from μ_{Collapse} ;
- (viii) transport status.

A suggested schematic record is:

```

iwasawa_calibration:
  id: "iw-cal-001"
  iwasawa_algebra: "Lambda"
  module: "M_Iw"
  arithmetic_mu:
    symbol: "mu_Iw"
    convention: "declared arithmetic convention"
    value_or_status: "known / unknown / conjectural / not_used"
  filtration:
    parameter: "cyclotomic_height / p_power_level / conductor_depth / congruence_depth"
    extraction_rule: "declared"
  realization:
    target: "FiltCh(k) / Perscons_k"
    core_readable: "yes / no / pending"
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F_Iw)"
    status: "not_invoked / computed / undefined / rejected"
  collapse_mu:
    symbol: "mu_Collapse"
    diagnostic_source: "kernel defect profile of phi_{i,W,tau}"
    status: "not_computed / computed / pending"
  transport:
    mu_Iw_to_mu_Collapse: "[Spec] / proved / rejected / not_claimed"
    mu_Collapse_to_mu_Iw: "[Spec] / proved / rejected / not_claimed"
  non_claim:
    - "mu_Iw is not identified with mu_Collapse by default."
    - "This calibration does not prove an Iwasawa main conjecture."

```

Remark 1.19 (Mandatory symbol separation). The calibration record should never write simply $\mu = 0$ when both μ_{Iw} and μ_{Collapse} are in scope. The subscript is mandatory.

1.7 C1.6. Congruence defect calibration

Definition 1.20 (Congruence defect calibration record). A *congruence defect calibration record* documents how congruence data are compared after realization and how any discrepancy is ledgered. It must record:

- (i) congruence source data;
- (ii) congruence relation;
- (iii) realization route;
- (iv) compared realized objects;

- (v) repaired evaluation comparison, if used;
- (vi) congruence defect;
- (vii) ledger entry;
- (viii) Type IV proxy status, if any;
- (ix) non-claim declarations.

A suggested schematic record is:

```

congruence_defect_calibration:
  id: "cong-cal-001"
  source_objects:
    - "A"
    - "A_prime"
  congruence_relation: "A congruent to A_prime modulo q / ideal / level"
  realization:
    R_cong_A: "F_A"
    R_cong_A_prime: "F_A_prime"
    target: "Perscons_k / FiltCh(k)"
  repaired_evaluation_comparison:
    left: "Eval_{i,W,tau}(F_A)"
    right: "Eval_{i,W,tau}(F_A_prime)"
    relation: "equal / interleaved / bounded_defect / failed / unknown"
  comparison:
    defect_symbol: "delta_cong"
    defect_value_or_bound: "declared"
  ledger:
    include_in_metric_ledger: true
    aggregation: "declared"
  typeIV_proxy:
    status: "none / advisory / candidate / computed_diagnostic"
    warning: "congruence failure is not Type IV diagnosis"
  non_claim:
    - "Congruence failure is not identified with Type IV obstruction by default."
    - "This calibration does not prove a congruence theorem."

```

Remark 1.21 (Congruence is not persistence equality). Arithmetic congruence does not automatically imply equality or interleaving of realized persistence objects. The realized comparison must be separately declared and supported.

1.8 C1.7. Arithmetic realization template

Definition 1.22 (Arithmetic realization calibration template). An *arithmetic realization calibration template* records the route by which arithmetic data become Core-readable. It must include:

- (i) source data;
- (ii) realization procedure;

- (iii) target category;
- (iv) filtration parameter;
- (v) constructibility check;
- (vi) window policy;
- (vii) threshold policy;
- (viii) monitored degrees;
- (ix) repaired evaluation policy;
- (x) purpose-faithfulness statement;
- (xi) artifact references.

A suggested schematic record is:

```

arithmetic_realization:
  id: "arith-real-001"
  source: "arith-src-001"
  procedure: "R_arith"
  target_category: "FiltCh(k) / Perscons_k / D[-1,0](k-mod)"
  filtration:
    parameter: "declared"
    direction: "increasing / decreasing"
    normalization: "declared / none"
  constructibility:
    status: "pass / reject / undefined"
    evidence: []
  windows:
    declared_windows: []
    MECE_status: "pass / reject / undefined"
  threshold_policy:
    tau: "declared"
    endpoint_policy: "declared"
  monitored_degrees: []
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F)"
    status: "not_applicable / candidate / computed / verified / failed"
  purpose_faithfulness:
    target_feature: "Selmer obstruction / congruence defect / tower growth / advisory"
    status: "[Spec] / proved / operational"
  artifacts: []
  non_claim:
    - "This realization does not prove the arithmetic target theorem."

```

Remark 1.23 (Realization can fail). A realization may fail because it is not constructible, not one-parameter, not purpose-faithful, not eligible, not compatible with repaired evaluation, or not artifact-supported. Failure should be recorded rather than hidden.

1.9 C1.8. Eligible regime calibration

Definition 1.24 (Eligible regime calibration record). An *eligible regime calibration record* documents whether a realized arithmetic object satisfies the requirements for invoking the Core Ext¹-edge. It must record:

- (i) admissible representative;
- (ii) compatibility with repaired evaluation;
- (iii) realization into $D^b(k\text{-mod})$;
- (iv) amplitude check;
- (v) edge-realization check;
- (vi) residual PH_1 object;
- (vii) Ext¹-clause status.

A suggested schematic record is:

```

eligible__regime__calibration:
  id: "elig-cal-001"
  input__object: "F"
  window: "W"
  threshold: "tau"
  admissible__representative:
    object: "C_{tau,W}F"
    status: "declared / constructed / failed"
  compatibility:
    required: "P_i(C_{tau,W}F) ~ = Eval_{i,W,tau}(F)"
    status: "pass / reject / undefined"
  realization:
    object: "R(C_{tau,W}F)"
    ambient__category: "D^b(k-mod)"
  amplitude:
    required: "[-1,0]"
    status: "pass / reject / undefined"
  edge__realization:
    required: "H^{-1}(R(C_{tau,W}F)) ~ = H_1^{\{tau,W\}}(F)"
    status: "pass / reject / undefined"
  ext__clause:
    allowed: "yes / no"
    reason: "declared"
  non__claim:
    - "Arithmetic Ext is not automatically Core Ext."
    - "Ext => PH is not used as a Core theorem."

```

Remark 1.25 (No eligibility, no Ext¹-edge). If admissible representative compatibility, amplitude, or edge realization fails, the Core Ext¹-edge cannot be invoked. The correct status is failure or pending, not informal acceptance.

1.10 C1.9. Arithmetic ledger template

Definition 1.26 (Arithmetic ledger calibration record). An *arithmetic ledger calibration record* documents how arithmetic transfer, comparison, congruence, normalization, discretization, tower, or bridge defects enter the δ -ledger.

A suggested schematic record is:

```
arithmetic_ledger:
  id: "arith-ledger-001"
  ledger_semantics:
    type: "additive / max / weighted / product / hybrid"
    strict_order: "declared"
  components:
    delta_arith:
      source: "arithmetic transfer"
      value_or_bound: "declared"
      artifact: []
    delta_cong:
      source: "congruence defect"
      value_or_bound: "declared"
      artifact: []
    delta_Iw:
      source: "Iwasawa-layer transfer"
      value_or_bound: "declared"
      artifact: []
    delta_tower:
      source: "tower comparison or limiting transfer"
      value_or_bound: "declared"
      artifact: []
    delta_bridge:
      source: "arithmetic bridge defect"
      value_or_bound: "declared"
      artifact: []
  aggregation:
    d_met: "declared metric-ledger aggregate"
    val_B_d_met: "computed_or_declared"
    protected_family: "B_family"
    gap_tau: "declared"
    comparison: "val_B(d_met) < gap_tau(B_family)"
    budget_verdict: "pass / reject / undefined / not_invoked"
  non_claim:
    - "Ledger pass does not prove arithmetic theorem."
    - "Ledger pass does not replace B-Gate+ or diagnostics."
```

Remark 1.27 (Ledger pass is not arithmetic pass). A ledger pass means only that the declared metric-ledger comparison has passed on its declared scope. It does not imply a Selmer statement, Iwasawa statement, congruence theorem, or arithmetic conclusion. Nor does it repair non-scalar obligations such as realization faithfulness, eligibility, terminal tameness, artifact identity, or claim-use permission.

1.11 C1.10. Type IV proxy versus monitored diagnostic

Definition 1.28 (Arithmetic Type IV proxy calibration). An *Arithmetic Type IV proxy calibration* records arithmetic signals that may suggest possible monitored Type IV obstruction. It must distinguish proxy status from diagnostic status.

A suggested schematic record is:

```
arithmetic_typeIV_proxy:
  id: "typeIV-proxy-001"
  source:
    type: "Iwasawa growth / congruence anomaly / tower control failure / limiting mismatch"
    description: "declared"
  proxy_status: "advisory / candidate / rejected"
  diagnostic_required:
    tower_artifact: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)"
    kernel_defect: "required"
    cokernel_defect: "required"
    terminal_generic_defect_dimensions: "required"
  monitored_diagnostic:
    mu_Collapse: "not_invoked / value / undefined"
    nu_Collapse: "not_invoked / value / undefined"
    verdict: "certified_zero / obstructed / undefined / not_invoked"
  non_claim:
    - "This proxy is not Type IV diagnosis."
    - "Arithmetic anomaly does not imply mu_Collapse or nu_Collapse nonzero."
```

Remark 1.29 (Proxy discipline). A proxy can tell us where to look. It cannot tell us what the monitored diagnostic is. Only the declared artifact

$$\phi_{i,W,\tau}$$

and its kernel/cokernel defect profiles determine the monitored Type IV diagnostic. The diagnostic record must use the full typed Type IV schema, including status not_invoked, undefined, certified_zero, or obstructed; missing data are not encoded as $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$.

1.12 C1.11. Arithmetic bridge calibration

Definition 1.30 (Arithmetic bridge calibration record). An *arithmetic bridge calibration record* documents a proposed or proved bridge between arithmetic meaning and Core-visible behavior. It must record the bridge direction, status, hypotheses, proof obligations, defects, and Core-facing payload if any.

A suggested schematic record is:

```
arithmetic_bridge:
  id: "arith-bridge-001"
  direction: "arithmetic_to_core / core_to_arithmetic"
  statement: "declared bridge statement"
  source_layer: "arithmetic"
  target_layer: "Core / interface / diagnostic"
  status: "[Spec] / bridge_candidate / partially_proved / bridge_theorem / rejected"
  core_payload:
```

```

repaired_evaluation: "Eval_{i,W,tau}(F) / none / pending"
representative: "C_{tau,W}F / none / pending"
tower_artifact: "phi_{i,W,tau} / none / pending"
diagnostics: "mu_Collapse, nu_Collapse / none / pending"
ledger: "val_B_d_met < gap_tau / none / pending"
hypotheses:
- id: "H1"
  statement: "declared"
  status: "proved / imported / [Spec] / unresolved"
required_lemmas:
- id: "L1"
  statement: "declared"
  status: "proved / pending / failed"
defects:
  delta_bridge: "value_or_bound / none / pending"
artifacts: []
non_claim:
- "Bridge Program is not bridge theorem."
- "Core pass does not imply arithmetic theorem unless this bridge is proved."

```

Remark 1.31 (Direction matters). The directions

$$\text{arithmetic} \Rightarrow \text{Core}$$

and

$$\text{Core} \Rightarrow \text{arithmetic}$$

are distinct. A calibration record must not merge them.

1.13 C1.12. Mirror and categorical arithmetic calibration

Definition 1.32 (Mirror/categorical calibration record). A *Mirror/categorical calibration record* documents how an arithmetic or motivic object is compared with a mirror-side or categorical object. It must record comparison cells, realization targets, repaired evaluation targets, and ledger costs.

A suggested schematic record is:

```

mirror_categorical_calibration:
  id: "mir-cal-001"
  source_domain: "arithmetic / motivic / geometric"
  mirror_domain: "categorical / derived / sheaf / Fukaya-side"
  source_object: "A"
  mirror_candidate: "A_dual"
  comparison:
    operation: "Mir / Comp"
    status: "strict / zero_cost / controlled / advisory / unavailable / failed"
  cells:
  - id: "alpha_Mir"
    type: "2-cell / higher-cell / correspondence"
    cost: "delta(alpha_Mir)"

```

realization:
 source_target: "Perscons_k / FiltCh(k) / technical extension"
 mirror_target: "Perscons_k / FiltCh(k) / technical extension"
 core_testable: "yes / no / pending"
 repaired_evaluation:
 source_eval: "Eval_{i,W,tau}(F_source) / none / pending"
 mirror_eval: "Eval_{i,W,tau}(F_mirror) / none / pending"
 comparison_status: "equal / controlled / failed / pending"
 ledger:
 delta_MT: "value_or_bound"
 included: true
 non_claim:
 - "Mirror comparison is not mirror symmetry theorem."
 - "Categorical comparison is not categorical equivalence theorem."

Remark 1.33 (Advisory comparison). An advisory mirror comparison may guide search. It may not support Core certification unless converted into strict, zero-cost, or controlled ledgered comparison.

1.14 C1.13. Fukaya-side calibration

Definition 1.34 (Fukaya calibration record). A *Fukaya calibration record* documents how a Fukaya-side structure is treated in an arithmetic or mirror-related interface. It is optional and always non-theorem unless supported by external theorems and a declared bridge.

A suggested schematic record is:

fukaya_calibration:
 id: "fuk-cal-001"
 fukaya_source:
 symplectic_object: "X"
 category: "Fuk(X)"
 auxiliary_data:
 coefficients: "declared"
 grading: "declared"
 brane_data: "declared"
 perturbation_data: "declared"
 extraction:
 route: "action_filtration / energy_filtration / microlocal_sheaf / other"
 target: "FiltCh(k) / Perscons_k"
 constructible: "pass / reject / undefined"
 repaired_evaluation:
 expression: "Eval_{i,W,tau}(F_Fuk)"
 status: "candidate / computed / verified / failed / pending"
 comparison:
 mirror_side: "declared"
 comparison_status: "strict / zero_cost / controlled / advisory / failed"
 delta_Fuk_Mir: "value_or_bound"
 non_claim:
 - "This calibration does not construct a Fukaya category."

- "This calibration does not prove homological mirror symmetry."
- "Fukaya obstruction is not Core obstruction by default."

1.15 C1.14. Failure-case catalog

Definition 1.35 (Arithmetic calibration failure). An *arithmetic calibration failure* occurs when a calibration record cannot support the intended interface use.

Common failure cases include:

- (F1) *Nonconstructible realization*. The arithmetic realization does not produce an object in $\text{Pers}_k^{\text{cons}}$.
- (F2) *Missing one-parameter filtration*. The source data have several parameters, but no declared one-parameter extraction.
- (F3) *Unproved purpose-faithfulness*. The realization is intuitive but not shown to preserve the relevant arithmetic feature.
- (F4) *Repaired evaluation failure*. The record claims a value of $\text{Eval}_{i,W,\tau}(F)$, but the supporting evaluation data are absent or inconsistent.
- (F5) *Eligibility failure*. The realized object is not in $D^{[-1,0]}(k\text{-mod})$, or the edge-realization condition fails.
- (F6) *μ -collision*. μ_{IW} and μ_{Collapse} are conflated.
- (F7) *Proxy inflation*. A congruence anomaly or Iwasawa growth signal is treated as monitored Type IV diagnosis.
- (F8) *Tower artifact failure*. A Type IV diagnostic is invoked without a declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

- (F9) *Unledgered transfer defect*. A realization or comparison introduces discrepancy but no ledger component.
- (F10) *Bridge inflation*. A Bridge Program or bridge candidate is treated as a bridge theorem.
- (F11) *Artifact insufficiency*. The record references artifacts that do not reconstruct the claim.
- (F12) *Arithmetic theorem inflation*. A Core pass is described as an arithmetic theorem without a proved Core-to-arithmetic bridge.

Remark 1.36 (Failure is useful). Calibration failure is not merely negative. It identifies which part of the interface requires repair.

1.16 C1.15. Non-claim examples

The following are recommended non-claim clauses for arithmetic calibration records.

non_claims:

- "This record is a calibration example, not a theorem."
- "This record does not prove BSD, ABC, RH, or any Iwasawa main conjecture."
- "Arithmetic realization is not assumed faithful unless separately proved."

- "Arithmetic Ext is not automatically Core Ext."
- " μ_Iw is not $\mu_Collapse$ by default."
- "Congruence failure is not Type IV diagnosis by default."
- "Core pass does not imply arithmetic conclusion without a proved transport theorem."
- "Search artifacts do not update Core verdicts."
- "AI confidence is not proof."
- "Ledger pass does not replace B-Gate+."
- "Ledger pass does not replace $Eval_ \{1, W, \tau\}(F)=0$."
- "Ledger pass does not replace monitored Type IV diagnostics."
- "A Type IV proxy does not replace $\phi_ \{i, W, \tau\}$."

Remark 1.37 (Non-claims should be copied into examples). Calibration examples should explicitly include non-claim fields. This reduces the risk that operational examples are later misread as theorem claims.

1.17 C1.16. Minimal arithmetic calibration manifest

Definition 1.38 (Minimal arithmetic calibration manifest). A *Minimal Arithmetic Calibration Manifest* is a structured record containing the minimum fields needed to audit an arithmetic calibration example. It contains:

- (i) calibration identifier;
- (ii) source data record;
- (iii) realization record;
- (iv) repaired evaluation record, if used;
- (v) admissible representative record, if used;
- (vi) window and threshold policy;
- (vii) constructibility status;
- (viii) eligibility status, if Ext^1 is invoked;
- (ix) ledger record;
- (x) Type IV proxy or diagnostic record;
- (xi) tower comparison artifact record, if diagnostics are invoked;
- (xii) bridge status, if any;
- (xiii) artifact references;
- (xiv) non-claim list;
- (xv) status label.

A compact schematic manifest is:

```

arithmetic_calibration_manifest:
  id: "arith-cal-manifest-001"
  status: "example / template / checked_calibration / failed_calibration"
  source: "arith-src-001"
  realization: "arith-real-001"
  repaired_evaluation:
    used: true
    record: "eval-record-001"
    expression: "Eval_{i,W,tau}(F)"
    status: "candidate / computed / verified / failed / pending"
  representative:
    used: false
    record: "C_{tau,W}F / none"
    status: "not_applicable / pass / fail / pending"
  windows: []
  threshold_policy: "declared"
  constructibility: "pass / reject / undefined"
  eligibility:
    used: false
    status: "not_applicable / pass / fail / pending"
  ledger: "arith-ledger-001"
  typeIV:
    proxy_record: "typeIV-proxy-001"
    tower_artifact: "phi_{i,W,tau} / none / linked / pending"
    monitored_diagnostic: "not_computed / linked / pending"
  bridge:
    status: "none / [Spec] / candidate / theorem / rejected"
    record: "arith-bridge-001"
  artifacts: []
  non_claims: []

```

Remark 1.39 (Calibration manifest versus Minimal Manifest). The Minimal Arithmetic Calibration Manifest is not the same as the Core Minimal Manifest of Appendix G. It is a companion-level operational manifest for examples and calibration records. A Core certificate still requires Appendix G.

1.18 C1.17. Recommended calibration workflow

A recommended arithmetic calibration workflow is:

- (C1) Declare arithmetic source data.
- (C2) Declare the intended interface purpose.
- (C3) Choose a realization route.
- (C4) Check one-parameter constructibility.
- (C5) Declare windows and thresholds.
- (C6) If Core-readable evaluation is used, record

$$\text{Eval}_{i,W,\tau}(F).$$

(C7) If representatives are invoked, record

$$C_{\tau, W}F.$$

(C8) If Ext^1 is used, check eligibility.

(C9) Record arithmetic transfer or comparison defects.

(C10) Build the arithmetic ledger.

(C11) Distinguish proxies from monitored diagnostics.

(C12) If monitored Type IV diagnostics are invoked, declare

$$\phi_{i, W, \tau}.$$

(C13) Declare bridge candidates and statuses.

(C14) Add non-claim clauses.

(C15) Attach artifacts.

(C16) Assign calibration status.

(C17) If moving toward certification, create a separate Core certificate record under Appendix G.

Remark 1.40 (Workflow boundary). The calibration workflow ends before Core certification. Moving from calibration to certification requires a separate certificate record and verifier-side checks.

1.19 C1.18. Summary of Arithmetic Calibration Companion

Proposition 1.41 (Companion 1 calibration package). *As an operational companion document, this companion provides the following calibration package.*

- (i) *Arithmetic calibration objects are examples, templates, or operational records, not theorems.*
- (ii) *Arithmetic source data should be declared with explicit purpose and non-claim fields.*
- (iii) *Cyclotomic tower calibration must distinguish arithmetic towers from Type IV diagnostic towers.*
- (iv) *Iwasawa calibration must distinguish μ_{Iw} from μ_{Collapse} .*
- (v) *Congruence calibration must distinguish congruence defect from monitored Type IV diagnosis.*
- (vi) *Arithmetic realization calibration must declare target category, filtration, constructibility, repaired evaluation, and purpose-faithfulness.*
- (vii) *Eligible regime calibration is required before invoking the Core Ext^1 -edge.*
- (viii) *Arithmetic ledger calibration must record defects, aggregation semantics, gap comparison, and non-claims.*
- (ix) *Type IV proxy calibration must distinguish proxy status from monitored diagnostic status.*
- (x) *Monitored Type IV diagnostic calibration requires the tower comparison artifact*

$$\phi_{i, W, \tau}.$$

- (xi) *Bridge calibration must distinguish bridge candidates, Bridge Programs, and bridge theorems.*
- (xii) *Mirror, categorical, and Fukaya-side calibration records are non-theorem interface support records.*
- (xiii) *Failure-case records should classify failures explicitly.*
- (xiv) *Non-claim clauses should be included in calibration examples.*
- (xv) *A Minimal Arithmetic Calibration Manifest supports audit of calibration examples but does not replace the Core Minimal Manifest.*

Proof. This theorem summarizes operational definitions and templates provided in this companion. Since the companion is non-theorem-bearing with respect to the AK Core, the proof is a structural verification of the companion's calibration roles rather than a mathematical theorem about arithmetic objects. $\square$

1.20 C1.19. Explicit exclusions

The following are not asserted in Companion 1.

- No arithmetic theorem is proved.
- No Birch–Swinnerton-Dyer statement is proved.
- No ABC statement is proved.
- No Riemann hypothesis statement is proved.
- No Iwasawa main conjecture is proved.
- No cyclotomic control theorem is proved.
- No congruence theorem is proved.
- No modularity theorem is proved.
- No mirror symmetry theorem is proved.
- No Fukaya-categorical theorem is proved.
- No arithmetic realization is assumed faithful by example.
- No calibration record is a Core certificate.
- No calibration record replaces

$$\text{Eval}_{1,w,\tau}(F) = 0.$$

- No calibration record replaces admissible representative records

$$C_{\tau,w}F.$$

- No calibration ledger pass is an arithmetic theorem.
- No arithmetic Type IV proxy is monitored Type IV diagnosis.

- No Type IV diagnostic is accepted without a declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No equality

$$\mu_{Iw} = \mu_{\text{Collapse}}$$

is asserted by calibration.

- No Core pass is interpreted as arithmetic theorem without a proved transport theorem.
- No Companion document enlarges the Core theorem package.
- No template field marked `not__applicable` or `not__used` becomes `not__invoked` without a declared scope-exclusion policy.
- No local companion defect token enters the Core metric budget without Appendix M catalog registration, support record, scalarization, and bottleneck discrepancy evidence.

1.21 C1.20. Provenance and status

This companion depends on:

- (i) Appendix MS-A: Arithmetic Interface Meta-Theory;
- (ii) Appendix MS-B: Cyclotomic / Iwasawa / Congruence Interfaces;
- (iii) Appendix MS-C: Mirror Reflection and Categorical Comparison;
- (iv) Appendix MS-F: Fukaya-Categorical Extension;
- (v) Appendix A: Repaired Barcode-Level Collapse and Constructible Persistence;
- (vi) Appendix B: Admissible Representatives;
- (vii) Appendix C: Eligible Realization Regime;
- (viii) Appendix D: Tower Diagnostics and Type IV Defect Calculus;
- (ix) Appendix G: Minimal Manifest and Proof Object Schema;
- (x) Appendix M: Ledger Semantics and Integrated Defect Catalog;
- (xi) Appendix U: Agent Semantics;
- (xii) Appendix W: Bridge Programs;
- (xiii) Appendix Z: Execution and Reproducibility Schema.

Its role is to support disciplined arithmetic interface operation without converting examples, templates, or calibration records into theorem claims.

Status labels for Companion 1. *Operational companion definitions:* Definitions [1.10](#), [1.11](#), [1.12](#), [1.14](#), [1.16](#), [1.18](#), [1.20](#), [1.22](#), [1.24](#), [1.26](#), [1.28](#), [1.30](#), [1.32](#), [1.34](#), [1.35](#), [1.38](#), [1.4](#), [1.5](#), [1.6](#).

Operational companion propositions: Propositions [1.7](#), [1.8](#), and [1.41](#).

Operational cautions: Remarks [1.1](#), [1.2](#), [1.3](#), [1.13](#), [1.15](#), [1.17](#), [1.19](#), [1.21](#), [1.23](#), [1.25](#), [1.27](#), [1.29](#), [1.31](#), [1.33](#), [1.36](#), [1.37](#), [1.39](#), [1.40](#), [1.9](#).

Explicit exclusions: Section C1.19.

Companion 2: Execution / Reproducibility Companion (v19.0.0 complete repair)

C2.1. Scope and non-proof status

This companion document provides operational templates and examples for execution, reproducibility, replay, proof-store records, and audit workflows in the AK framework. It is a companion document, not a theorem-bearing appendix. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this companion is:

Execution and reproducibility templates are operational aids, not proofs.

This companion supports the practical use of:

Appendix G, Appendix Y, Appendix Z.

It provides example schemas for:

`run.yaml`, `artifact_manifest.json`, `eval.json`, `ledger.json`, `diagnostics.json`, `gate.json`, `proof_store_entry`.

No companion template, execution package, replay record, audit checklist, stored object, or reproducibility manifest may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \\ (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}).$$

Remark 1.42 (Companion status). This document is operational and illustrative. It may define recommended schemas and checklists. It does not certify any proof object by itself.

Remark 1.43 (Relation to Appendix Z). Appendix Z defines the execution and reproducibility schema formally. This companion provides practical templates for instantiating that schema. When in doubt, Appendix Z takes precedence.

Remark 1.44 (Final companion boundary). This is the final operational companion. Its purpose is to make the AK infrastructure executable and auditable without confusing execution, replay, storage, or audit with proof.

C2.1bis. v19 execution hardening and atomic status boundary

Definition 1.45 (Execution-sovereignty boundary). The execution-sovereignty boundary is the rule that successful execution, replay, storage, hash validation, manifest completion, audit-pass status, or reproducibility level does not change a Core, interface, or bridge verdict unless the output is converted into the declared verifier-side record and passes the applicable verifier-side protocol.

Definition 1.46 (Execution-to-verifier conversion record). An execution-to-verifier conversion record declares the execution output, target verifier-side schema, scope, environment, artifact identities, authority, hypotheses, result status *pass/reject/undefined*, and typed failure route. It is required whenever an execution artifact, replay result, proof-store object, or audit record is consumed as evidence.

Definition 1.47 (Replay atomic-status mapping). Replay results map to atomic statuses only as follows: replay-pass may support pass for the declared replay-support criterion only; replay-fail and replay-divergent map to reject for that replay-support criterion; replay-incomplete and replay-blocked map to undefined; and replay-not-applicable maps to not_*_invoked* only under a declared scope-exclusion policy, otherwise to undefined for theorem-facing use. No replay result by itself proves theorem validity.

Definition 1.48 (Execution defect typing). Execution adequacy, environment compatibility, artifact identity, replay divergence, and proof-store status are normally status-ledger or qualitative-governance components. They are not metric-admissible unless a separate support record converts them into a bottleneck discrepancy bound under the Appendix M ledger catalog.

Proposition 1.49 (Replay and audit are support checks, not theorem checks by default). *A replay-pass or audit-pass result verifies only the declared execution or audit-support criterion. It does not prove the mathematical claim interpreted from that execution unless the required verifier-side proof or conversion record is present.*

Proof. Replay checks reconstructibility under a declared environment and equivalence criterion. Audit checks completeness and consistency of records. Theorem validity requires the corresponding mathematical proof, diagnostic, gate, ledger, and claim-use obligations. $\square$

Remark 1.50 (Execution status-token discipline). Template statuses such as verified, audit-passed, replay-passed, complete, and adequate are workflow statuses unless explicitly mapped to Chapter 7 atomic statuses by a scoped conversion record. They do not replace Appendix U agent-output labels, Appendix Y proof-store statuses, Appendix Z replay statuses, Chapter 1 claim roles, or CR-v19 invocation governance.

C2.2. Execution package overview

Definition 1.51 (Execution package). An *execution package* is a collection of files and records sufficient to reconstruct a declared execution run. A recommended execution package contains:

- (i) run.yaml;
- (ii) artifact_manifest.json;
- (iii) eval.json, if repaired evaluation is computed or checked;
- (iv) ledger.json, if a budget check is used;
- (v) diagnostics.json, if diagnostics are computed;
- (vi) gate.json, if a Core gate is checked;

- (vii) proof_store_entry.json;
- (viii) reproducibility_manifest.json;
- (ix) logs;
- (x) environment records;
- (xi) referenced artifacts.

Definition 1.52 (Execution package status). An execution package has one of the following statuses:

- (i) draft;
- (ii) incomplete;
- (iii) replay-ready;
- (iv) replay-passed;
- (v) replay-failed;
- (vi) audit-ready;
- (vii) audit-passed;
- (viii) audit-failed;
- (ix) blocked;
- (x) superseded;
- (xi) rejected.

Remark 1.53 (Package status is not theorem status). An execution package may be replay-passed or audit-passed while the underlying mathematical claim remains unproved. The package status and claim status must remain distinct.

C2.3. run.yaml template

Definition 1.54 (run.yaml operational template). The run.yaml template records the scope, inputs, parameters, commands, environment, outputs, verification criteria, and replay instructions of an execution run.

A recommended template is:

run:

id: "run-0001"

title: "AK execution run"

scope:

type: "persistence_compute / repaired_evaluation / representative_check / tower_artifact_check / diagnostics /

description: "declared run scope"

status: "draft / replay-ready / replay-passed / replay-failed / audit-ready / audit-passed / blocked"

inputs:

artifacts:

- id: "artifact-input-001"
- role: "input"
- path_or_uri: "path/to/input"
- hash: "sha256:..."
- required: true

parameters:

- field: "k"
- window_policy:
 - windows:
 - id: "W1"
 - interval: "[a,a']"
 - status: "declared"
- threshold_policy:
 - tau: "declared_value"
 - endpoint_policy: "declared"
- monitored_degrees:
 - 0
 - 1
- random_seed:
 - used: false
 - value: null

core_repaired_objects:

- repaired_evaluation:
 - used: true
 - expression: "Eval_{i,W,tau}(F) = T_bar_tau(W_W P_i(F))"
 - eval_record: "eval.json"
- admissible_representative:
 - used: false
 - expression: "C_{tau,W}F"
 - representative_record: "artifact-or-record-id"
- tower_artifact:
 - used: false
 - expression: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)"
 - artifact_record: "artifact-or-record-id"

environment:

- environment_id: "env-0001"
- os: "declared"
- container: "image_digest_or_none"
- language_versions:
 - python: "version_or_none"
 - coq: "version_or_none"
 - lean: "version_or_none"
- dependencies:
 - lockfile: "path/to/lockfile"
 - hash: "sha256:..."

commands:

- id: "cmd-001"
description: "command description"
command: "exact command or workflow step"
expected_output: "declared"

outputs:

- artifacts:
- id: "artifact-output-001"
role: "output"
path_or_uri: "path/to/output"
hash_policy: "required / optional / none"

verification:

- criteria:
- "declared criterion"
- authority: "declared tool or agent"
expected_result: "pass / reject / undefined"

replay:

- instructions: "steps to replay"
equivalence_criterion: "exact_hash / numerical_tolerance / proof_assistant_acceptance / diagnostic_equality /"
tolerance: null

non_claims:

- "Successful execution is not proof by itself."
- "Replay pass is not theorem validity."
- "Execution artifacts do not replace Core verifier conditions."

Remark 1.55 (Run scope must be narrow). A run should have a clear scope. A run that computes, verifies, audits, and interprets theorem-level meaning without separation is difficult to audit.

C2.4. artifact_manifest.json template

Definition 1.56 (artifact_manifest.json operational template). The artifact_manifest.json template records all artifacts used or produced by an execution run.

A recommended template is:

```
{
  "artifact_manifest": {
    "id": "artifact-manifest-0001",
    "associated_run": "run-0001",
    "status": "draft / complete / incomplete / audit-ready / superseded",
    "artifacts": [
      {
        "id": "artifact-001",
        "role": "input / output / intermediate / log / reference",
```



```

"type": "data / code / proof_script / barcode / eval / representative / tower_artifact / diagram / ledger / diagram",
"path_or_uri": "path/to/artifact",
"version": "v1",
"hash": "sha256:...",
"hash_required": true,
"producer_run": "run-0000",
"consumer_runs": ["run-0001"],
"supports": [
"Eval_{i,W,tau}(F) / C_{tau,W}F / phi_{i,W,tau} / ledger / diagnostics / gate / proof_store_entry"
],
"adequacy_status": "unchecked / adequate / inadequate / pending",
"audit_notes": []
}
],
"non_claims": [
"Artifact availability is not artifact adequacy.",
"Artifact manifest completeness is not proof."
]
}
}
}

```

Remark 1.57 (Adequacy must be checked). The manifest records existence and identity. A separate adequacy check is required to confirm whether the artifact supports the cited claim or computation.

C2.5. eval.json template

Definition 1.58 (eval.json operational template). The eval.json template records repaired evaluation computations:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

A recommended template is:

```

{
"evaluation": {
"id": "eval-0001",
"associated_run": "run-0001",
"input_object": "F",
"degree": 1,
>window": {
"id": "W1",
"interval": "[a,a'",
"policy": "declared"
},
"threshold": {
"tau": "declared_value",
"endpoint_policy": "declared"
},
"persistence_extraction": {
"method": "declared",

```

```

    "target": "Perscons_k",
    "constructibility_status": "pass / reject / undefined"
  },
  "repaired_evaluation": {
    "expression": "Eval_{i,W,tau}(F) = T_bar_tau(W_W P_i(F))",
    "barcode_policy": "declared",
    "window_restriction_policy": "declared",
    "threshold_deletion_policy": "delete bars <= tau",
    "result_profile": "artifact-or-description",
    "vanishing_result": "zero / nonzero / undefined / rejected"
  },
  "artifacts": ["artifact-001"],
  "verification": {
    "status": "candidate / computed / verified / rejected / undefined",
    "authority": "declared"
  },
  "non_claims": [
    "The existence of eval.json does not imply Eval correctness.",
    "Eval summary is not audit-sufficient without reconstructible data."
  ]
}

```

Remark 1.59 (Evaluation record is now explicit). This template is the operational replacement for older exact-collapse execution wording. Execution records should refer to $\text{Eval}_{i,W,\tau}$, not to a global exact collapse operation.

C2.6. ledger.json template

Definition 1.60 (ledger.json operational template). The ledger.json template records ledger components, aggregation semantics, gap values, and budget verdict states.

A recommended template is:

```

{
  "ledger": {
    "id": "ledger-0001",
    "associated_run": "run-0001",
    "ledger_semantics": {
      "type": "additive / max / weighted / product / hybrid",
      "identity": "0 or declared",
      "strict_order": "<",
      "aggregation_rule": "declared"
    },
    "components": [
      {
        "id": "delta-001",
        "symbol": "delta_disc",
        "source": "discretization / measurement / normalization / bridge / categorical_comparison / arithmetic_transf

```

```

    "value": "declared numeric or symbolic bound",
    "unit": "declared_or_none",
    "artifact_reference": "artifact-001",
    "status": "declared / computed / bounded / undefined"
  }
],
"aggregate": {
  "val_B_d_met": "computed_or_declared",
  "gap_tau": "declared",
  "comparison": "val_B(d_met) < gap_tau(B_family)",
  "result": "pass / reject / undefined / not_invoked"
},
"audit": {
  "replayable": true,
  "notes": []
},
"non_claims": [
  "The existence of ledger.json does not imply budget pass.",
  "Budget pass does not replace B-Gate+ or diagnostics."
]
}
}

```

Remark 1.61 (Strict comparison). The strict inequality

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B})$$

must be recorded as strict. A non-strict or ambiguous comparison should not be accepted as an Audit Mode pass.

C2.7. diagnostics.json template

Definition 1.62 (diagnostics.json operational template). The diagnostics.json template records diagnostic computations, especially monitored Type IV diagnostics.

A recommended template is:

```

{
  "diagnostics": {
    "id": "diagnostics-0001",
    "associated_run": "run-0001",
    "diagnostic_type": "TypeIV / PH / Ext / tower / advisory",
    "monitored_degrees": [0, 1],
    "windows": [
      {
        "id": "W1",
        "interval": "[a,a')"
      }
    ]
  },
}

```

```

"thresholds": [
  {
    "tau": "declared_value"
  }
],
"tower_comparisons": [
  {
    "id": "tower-comp-001",
    "degree": 1,
    "window": "W1",
    "threshold": "tau",
    "tower_artifact": "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)",
    "artifact_status": "candidate / declared / verified / rejected / undefined",
    "source_profile": "ColimProfile_{i,W,tau}(F_bullet)",
    "target_profile": "ApexProfile_{i,W,tau}(F_infty)",
    "kernel_defect_object": "artifact-or-description",
    "cokernel_defect_object": "artifact-or-description",
    "terminal_generic_defect_dimensions": {
      "mu_component": "value_or_undefined",
      "nu_component": "value_or_undefined"
    },
    "verdict": "certified_zero / obstructed / undefined / not_invoked"
  }
],
"summary": {
  "mu_Collapse": "value_or_undefined",
  "nu_Collapse": "value_or_undefined",
  "diagnostic_verdict": "certified_zero / obstructed / undefined / not_invoked"
},
"artifact_references": [],
"non_claims": [
  "Diagnostic summary is insufficient without reconstructible computation.",
  "A Type IV proxy is not monitored Type IV diagnosis.",
  "A monitored diagnostic requires a declared phi_{i,W,tau} artifact."
]
}
}

```

Remark 1.63 (Tower artifact must be reconstructible). For monitored Type IV diagnostics, the tower comparison artifact

$$\phi_{i,W,\tau}$$

and its defect objects must be reconstructible. A summary value alone is not audit-sufficient. The execution template must not use a zero pair as a substitute for the full typed Type IV diagnostic record.

C2.8. gate.json template

Definition 1.64 (gate.json operational template). The gate.json template records Core gate checks and clause-level verdicts.

A recommended template is:

```
{
  "gate": {
    "id": "gate-0001",
    "associated_run": "run-0001",
    "gate_type": "B-Gate+ / CollapseAdmissible / interface_pre_gate / other",
    "input_object": "object-id",
    "window": "W1",
    "threshold": "tau",
    "clauses": [
      {
        "id": "clause-001",
        "statement": "Eval_{1,W,tau}(F) = 0",
        "result": "pass / reject / undefined",
        "evidence": ["eval-0001"]
      },
      {
        "id": "clause-002",
        "statement": "Ext1 = 0 in eligible regime using C_{tau,W}F",
        "result": "pass / reject / undefined / not_invoked",
        "evidence": ["artifact-002"]
      },
      {
        "id": "clause-003",
        "statement": "Type IV diagnostics clean using phi_{i,W,tau}",
        "result": "pass / reject / undefined",
        "evidence": ["diagnostics-0001"]
      },
      {
        "id": "clause-004",
        "statement": "budget condition val_B(d_met) < gap_tau(B_family)",
        "result": "pass / reject / undefined",
        "evidence": ["ledger-0001"]
      }
    ],
    "final_verdict": "pass / reject / undefined / blocked",
    "verifying_authority": "declared",
    "non_claims": [
      "The existence of gate.json does not imply gate pass.",
      "Clause-level support is required."
    ]
  }
}
```

Remark 1.65 (Gate pass must not be asserted globally). A gate pass is scoped to the declared object, window, threshold, and clauses. It should not be generalized without a separate theorem.

C2.9. proof_store_entry.json template

Definition 1.66 (proof_store_entry.json operational template). The proof_store_entry.json template records an object stored in the Proof Store and its status metadata.

A recommended template is:

```
{
  "proof_store_entry": {
    "id": "pse-0001",
    "object_type": "search_artifact / candidate / formal_fragment / verified_fragment / certificate_candidate / con",
    "title": "declared title",
    "content_reference": "path_or_uri_or_hash",
    "version": "v1",
    "status": "draft / candidate / verified / rejected / superseded / [Spec]",
    "claim_type": "Core theorem / Core lemma / operational policy / interface theorem / bridge candidate / bridge th",
    "core_facing_payload": {
      "Eval": "Eval_{i,W,tau}(F) / none / pending",
      "representative": "C_{tau,W}F / none / pending",
      "tower_artifact": "phi_{i,W,tau} / none / pending",
      "diagnostics": "mu_Collapse, nu_Collapse / none / pending",
      "ledger": "val_B(d_met) < gap_tau(B_family) / none / pending"
    },
    "dependencies": [
      {
        "id": "dep-001",
        "type": "logical / artifact / ledger / diagnostic / bridge / advisory",
        "target": "pse-or-artifact-id",
        "status": "discharged / pending / failed / advisory"
      }
    ],
    "artifacts": ["artifact-001"],
    "verification_records": ["verification-001"],
    "audit_records": ["audit-001"],
    "provenance": {
      "created_by": "agent-or-human",
      "created_at": "timestamp-or-version",
      "modified_by": [],
      "status_history": []
    },
    "non_claims": [
      "Storage is not certification.",
      "Proof Store entry does not imply theorem status."
    ]
  }
}
```

Remark 1.67 (Storage status versus claim status). The object may be stored successfully while the mathematical claim remains unverified. Storage and claim status must remain separated.

C2.10. reproducibility_manifest.json template

Definition 1.68 (reproducibility_manifest.json operational template). The reproducibility_manifest.json template links run schema, artifacts, environment, repaired evaluation records, ledgers, diagnostics, gates, replay results, and audit records.

A recommended template is:

```
{
  "reproducibility_manifest": {
    "id": "repro-manifest-0001",
    "associated_run": "run-0001",
    "run_yaml": "run.yaml",
    "artifact_manifest": "artifact_manifest.json",
    "environment": {
      "environment_id": "env-0001",
      "reference": "environment.lock / container_digest / proof_assistant_env"
    },
    "records": {
      "eval": "eval.json / not_applicable",
      "ledger": "ledger.json / not_applicable",
      "diagnostics": "diagnostics.json / not_applicable",
      "gate": "gate.json / not_applicable",
      "proof_store_entry": "proof_store_entry.json"
    },
    "replay": {
      "result": "replay-pass / replay-fail / replay-incomplete / replay-divergent / replay-not-
applicable / replay-blocked",
      "equivalence_criterion": "exact_hash / tolerance / formal_acceptance / diagnostic_equality / semantic",
      "replay_log": "path/to/replay.log"
    },
    "reproducibility_level": "R0 / R1 / R2 / R3 / R4 / R5",
    "audit": {
      "result": "audit-pass / audit-fail / audit-incomplete / audit-blocked",
      "scope": "declared",
      "audit_record": "audit-001"
    },
    "non_claims": [
      "Reproducibility manifest is not proof.",
      "Replay pass is not theorem validity."
    ]
  }
}
```

Remark 1.69 (Top-level connector). The reproducibility manifest is the top-level connector for execution audit. It should not duplicate all records, but it should link them unambiguously.

C2.11. Verification record template

Definition 1.70 (Verification record template). A *verification record template* records the result of a scoped verification action.

A recommended template is:

```
{
  "verification_record": {
    "id": "verification-001",
    "object": "pse-0001",
    "criteria": [
      "declared criterion 1",
      "declared criterion 2"
    ],
    "authority": {
      "type": "human / AI / proof_assistant / computation_engine / audit_tool",
      "name": "declared",
      "scope": "declared"
    },
    "environment": "env-0001",
    "result": "pass / fail / inconclusive / blocked / not_applicable",
    "evidence": ["artifact-001"],
    "logs": ["verification.log"],
    "core_conditions_checked": {
      "Eval_{i,W,tau}": "yes / no / not_applicable",
      "C_{tau,W}F": "yes / no / not_applicable",
      "phi_{i,W,tau}": "yes / no / not_applicable",
      "diagnostics": "yes / no / not_applicable",
      "ledger": "yes / no / not_applicable",
      "gate": "yes / no / not_applicable"
    },
    "non_claims": [
      "Verification result is scoped to declared criteria.",
      "Verification pass does not imply unrelated claims."
    ]
  }
}
```

C2.12. Audit record template

Definition 1.71 (Audit record template). An *audit record template* records whether a proof object, certificate, or execution package is reconstructible within a declared scope.

A recommended template is:

```
{
  "audit_record": {
    "id": "audit-001",
    "scope": {
```



```

    "objects": ["pse-0001"],
    "runs": ["run-0001"],
    "artifacts": ["artifact-001"],
    "records": ["eval.json", "ledger.json", "diagnostics.json", "gate.json"]
  },
  "checks": [
    {
      "id": "audit-check-001",
      "description": "artifact hash consistency",
      "result": "pass / reject / undefined"
    },
    {
      "id": "audit-check-002",
      "description": "manifest completeness",
      "result": "pass / reject / undefined"
    },
    {
      "id": "audit-check-003",
      "description": "replayability",
      "result": "pass / reject / undefined"
    },
    {
      "id": "audit-check-004",
      "description": "Core-facing record reconstructibility",
      "result": "pass / reject / undefined"
    }
  ],
  "result": "audit-pass / audit-fail / audit-incomplete / audit-blocked",
  "notes": [],
  "non_claims": [
    "Audit pass is not mathematical proof by itself.",
    "Audit checks reconstructibility and record integrity."
  ]
}

```

C2.13. R0–R5 reproducibility checklist

Definition 1.72 (Reproducibility checklist). A *reproducibility checklist* is an operational checklist for assigning reproducibility level R0–R5.

Level R0 checklist.

- Result is stated.
- Execution schema is missing or insufficient.
- Required artifacts are missing or unavailable.
- Replay is not possible.

Level R1 checklist.

- Artifacts are referenced.
- Some metadata exists.
- Run cannot be replayed from available records.

Level R2 checklist.

- Inputs are recorded.
- Parameters are partially or fully recorded.
- Outputs are recorded.
- Environment or commands are incomplete.

Level R3 checklist.

- Inputs, outputs, parameters, and commands are recorded.
- Compatible environment is available.
- Replay succeeds under declared tolerance or equivalence.

Level R4 checklist.

- Captured environment is available.
- Artifact hashes match.
- Exact or content-identical replay succeeds.
- Logs and outputs are stable.

Level R5 checklist.

- R4 conditions hold.
- Independent audit is recorded.
- Proof Store entries are linked.
- Repaired evaluation records are linked where applicable.
- Ledger, diagnostics, and gate records are linked where applicable.
- Manifest completeness is checked.
- Audit result is audit-pass.

Remark 1.73 (R5 is not theorem validity). R5 is a high reproducibility and audit level. It is still not theorem validity unless the proof content is valid.

C2.14. Replay procedure example

A recommended replay procedure is:

- (R1) Retrieve `reproducibility__manifest.json`.
- (R2) Retrieve `run.yaml`.
- (R3) Retrieve `artifact__manifest.json`.
- (R4) Check required input artifact availability.
- (R5) Verify hashes for all hash-bound artifacts.
- (R6) Reconstruct the declared environment.
- (R7) Execute commands in declared order.
- (R8) Compare outputs under the declared replay equivalence criterion.
- (R9) Recompute or inspect `eval.json`, if applicable.
- (R10) Recompute or inspect `ledger.json`, if applicable.
- (R11) Recompute or inspect `diagnostics.json`, if applicable.
- (R12) Recompute or inspect `gate.json`, if applicable.
- (R13) Record replay result.
- (R14) Update reproducibility level if appropriate.
- (R15) If replay fails, create a reproducibility failure record.

Remark 1.74 (Replay should not update claim status automatically). Replay pass may support audit status, but it should not automatically promote the mathematical claim. Promotion requires the relevant evidence and authority.

C2.15. Failure and demotion examples

Definition 1.75 (Execution failure example). An *execution failure example* is an operational example showing when an execution package should be reviewed, demoted, or rejected.

Common examples include:

- (E1) *Missing artifact*. A required input artifact is absent. Recommended status: replay-blocked.
- (E2) *Hash mismatch*. A retrieved artifact does not match its declared hash. Recommended status: audit-blocked until resolved.
- (E3) *Environment unavailable*. The required environment cannot be reconstructed. Recommended status: R2 or lower.
- (E4) *Parameter ambiguity*. Threshold τ , window W , monitored degree i , or ledger semantics are missing. Recommended status: incomplete.

(E5) *Repaired evaluation inconsistency*. eval.json states

$$\text{Eval}_{i,w,\tau}(F) = 0$$

but the barcode, window, threshold, or deletion policy is missing. Recommended status: evaluation-incomplete.

(E6) *Representative ambiguity*. $C_{\tau,w}F$ is invoked, but no admissible representative record is attached. Recommended status: representative-incomplete.

(E7) *Ledger inconsistency*. $\text{val}_B(\mathbf{d}_{\text{met}})$ in ledger.json does not match component aggregation. Recommended status: verification-fail or audit-fail.

(E8) *Diagnostic summary without computation*. μ_{Collapse} and u_{Collapse} are stated, but $\phi_{i,w,\tau}$ or defect objects are missing. Recommended status: diagnostic-incomplete.

(E9) *Gate verdict without clauses*. gate.json states pass, but clause-level evidence is missing. Recommended status: gate-incomplete.

(E10) *Replay divergence*. Replayed outputs fail the declared equivalence criterion. Recommended status: replay-fail.

(E11) *AI-generated proof object without status label*. Recommended status: non-verdict until labeled and reviewed.

(E12) *Stored object promoted without evidence*. Recommended status: demote to previous supported status.

Remark 1.76 (Demotion protects integrity). Demotion is not punitive. It preserves the reliability of the proof infrastructure.

C2.16. Minimal execution package manifest

Definition 1.77 (Minimal execution package manifest). A *Minimal Execution Package Manifest* is the minimum operational record needed to audit an execution package. It contains:

- (i) execution package identifier;
- (ii) run identifier;
- (iii) run scope;
- (iv) run.yaml reference;
- (v) artifact manifest reference;
- (vi) environment reference;
- (vii) repaired evaluation reference, if applicable;
- (viii) ledger reference, if applicable;
- (ix) diagnostics reference, if applicable;
- (x) gate reference, if applicable;

- (xi) proof-store entry reference;
- (xii) replay result;
- (xiii) reproducibility level;
- (xiv) audit result;
- (xv) non-claim list.

A compact schematic manifest is:

```
minimal_execution_package_manifest:
  id: "exec-package-0001"
  status: "draft / replay-ready / replay-passed / audit-ready / audit-passed / failed"
  run: "run-0001"
  run_scope: "declared"
  run_yaml: "run.yaml"
  artifact_manifest: "artifact_manifest.json"
  environment: "env-0001"
  records:
    eval: "eval.json / not_applicable"
    ledger: "ledger.json / not_applicable"
    diagnostics: "diagnostics.json / not_applicable"
    gate: "gate.json / not_applicable"
    proof_store_entry: "proof_store_entry.json"
  replay:
    result: "replay-pass / replay-fail / replay-incomplete / replay-blocked"
    level: "R0 / R1 / R2 / R3 / R4 / R5"
  audit:
    result: "audit-pass / audit-fail / audit-incomplete / audit-blocked"
  non_claims:
    - "Execution package is not proof by itself."
    - "Reproducibility supports audit but does not prove theorem validity."
```

Remark 1.78 (Operational manifest versus Core manifest). The Minimal Execution Package Manifest does not replace the Core Minimal Manifest of Appendix G. It supports execution audit. Core certification still requires verifier-side Core data.

C2.17. Recommended execution workflow

A recommended execution and reproducibility workflow is:

- (X1) Define the run scope.
- (X2) Create run.yaml.
- (X3) Register input artifacts.
- (X4) Record environment information.
- (X5) Execute the run.

- (X6) Store output artifacts.
- (X7) Create or update artifact_manifest.json.
- (X8) If repaired evaluation is used, create eval.json.
- (X9) If budget is used, create ledger.json.
- (X10) If diagnostics are used, create diagnostics.json.
- (X11) If gates are checked, create gate.json.
- (X12) Register the object in the Proof Store.
- (X13) Create reproducibility_manifest.json.
- (X14) Attempt replay.
- (X15) Assign reproducibility level R0–R5.
- (X16) Perform audit.
- (X17) If failure is detected, demote or block the relevant object.
- (X18) If all required evidence is present, prepare for verifier-side certification.

Remark 1.79 (Workflow boundary). This workflow prepares execution and audit support. It does not itself prove the mathematical claim.

C2.18. Non-claim examples

Recommended non-claim clauses for execution and reproducibility records are:

non_claims:

- "This execution record is not proof by itself."
- "Successful execution does not imply certificate success."
- "Replay pass does not imply theorem validity."
- "Audit pass checks reconstructibility, not mathematical truth by itself."
- "Storage is not certification."
- "Hash match proves content identity, not validity."
- "eval.json existence does not imply Eval correctness."
- "ledger.json existence does not imply budget pass."
- "diagnostics.json existence does not imply Type IV cleanliness."
- "gate.json existence does not imply gate pass."
- "Formal replay is scoped to the checked formal statement."
- "Numerical reproducibility does not imply continuum validity."
- "No execution artifact replaces B-Gate+."
- "No execution artifact replaces $\text{Eval}_{\{1, W, \tau\}}(F)=0$."
- "No execution artifact replaces $C_{\{\tau, W\}}F$ when representatives are invoked."
- "No execution artifact replaces $\phi_{\{i, W, \tau\}}$ when Type IV diagnostics are invoked."
- "No execution artifact replaces monitored Type IV diagnostics."
- "No execution artifact replaces the budget condition."

C2.19. Summary of Execution / Reproducibility Companion

Proposition 1.80 (Companion 2 execution-reproducibility package). *As an operational companion document, this companion provides the following package.*

- (i) *Execution packages organize run schema, artifacts, repaired evaluations, ledgers, diagnostics, gates, proof-store entries, and reproducibility manifests.*
- (ii) *run.yaml records run scope, inputs, parameters, Core-facing object configuration, environment, commands, outputs, and replay instructions.*
- (iii) *artifact__manifest.json records artifact identity and role but does not prove artifact adequacy.*
- (iv) *eval.json records repaired evaluation computations but does not imply evaluation correctness by existence.*
- (v) *ledger.json records budget components and comparison but does not imply budget pass by existence.*
- (vi) *diagnostics.json records diagnostic computations but does not imply Type IV cleanliness by existence.*
- (vii) *gate.json records gate clauses and verdicts but does not imply gate pass by existence.*
- (viii) *proof__store__entry.json records stored object metadata but storage is not certification.*
- (ix) *reproducibility__manifest.json links execution records for audit.*
- (x) *Verification and audit records must be scoped.*
- (xi) *R0–R5 reproducibility levels classify replay and audit support, not theorem validity.*
- (xii) *Replay procedures should be explicit and should not automatically promote mathematical claim status.*
- (xiii) *Failure cases should trigger review, blocking, or demotion where necessary.*
- (xiv) *Minimal Execution Package Manifest supports execution audit but does not replace the Core Minimal Manifest.*
- (xv) *Non-claim clauses should be included in execution and reproducibility records.*

Proof. This theorem summarizes operational templates and checklists provided in this companion. Since the companion is non-theorem-bearing with respect to the AK Core, the proof is a structural verification of the companion’s execution and reproducibility roles rather than a mathematical theorem about Core validity. $\square$

C2.20. Explicit exclusions

The following are not asserted in Companion 2.

- No execution package is proof by itself.
- No run.yaml file is proof by itself.
- No artifact manifest proves artifact adequacy.
- No eval.json file proves repaired evaluation correctness by existence.

- No ledger.json file proves budget pass by existence.
- No diagnostics.json file proves Type IV cleanliness by existence.
- No gate.json file proves gate pass by existence.
- No Proof Store entry certifies an object by storage.
- No reproducibility manifest proves theorem validity.
- No replay-pass result proves a theorem by itself.
- No audit-pass result proves mathematical truth by itself.
- No R5 reproducibility level implies theorem validity.
- No hash match proves validity.
- No formal replay proves informal interpretation without an interpretation theorem.
- No numerical reproducibility proves continuum validity.
- No execution artifact replaces B-Gate⁺.
- No execution artifact replaces the repaired topological condition

$$\text{Eval}_{1,w,\tau}(F) = 0.$$

- No execution artifact replaces admissible representative records

$$C_{\tau,w}F.$$

- No execution artifact replaces tower comparison artifacts

$$\phi_{i,w,\tau}.$$

- No execution artifact replaces monitored Type IV diagnostics.
- No execution artifact replaces the budget condition

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}(\mathfrak{B}).$$

- No Companion document enlarges the Core theorem package.
- No replay-not-applicable result becomes not_invoked without a declared scope-exclusion policy.
- No environment, identity, adequacy, replay, or storage defect is compensated by metric slack unless it is separately converted into a metric-admissible component under Appendix M.

C2.21. Final companion closure

Proposition 1.81 (Companion closure principle). *Companion 1 and Companion 2 provide operational calibration, execution, reproducibility, and audit templates for the AK framework. They do not add mathematical theorems, do not modify the Core verifier, do not replace the Problem Interface appendices, and do not promote examples, records, or templates to theorem status.*

Proof. Companion 1 is an arithmetic calibration document. Companion 2 is an execution and reproducibility document. Both documents provide operational templates and non-claim examples. Neither supplies new Core axioms, new Core theorems, new bridge theorems, or theorem-level problem conclusions. Therefore the companion layer is operationally closed but not theorem-expanding. $\square$

Remark 1.82 (Final operational closure). After this companion, the v19.0.0 repaired document set has a closed operational shell: arithmetic calibration is handled by Companion 1, and execution/reproducibility is handled by Companion 2. All theorem-level claims must still pass through the appropriate Core, bridge, interface, and audit disciplines.

C2.22. Provenance and status

This companion depends on:

- (i) Appendix G: Minimal Manifest and Proof Object Schema;
- (ii) Appendix Y: AI Platform and Proof Store;
- (iii) Appendix Z: Execution and Reproducibility Schema;
- (iv) Appendix A: Repaired Barcode-Level Collapse and Constructible Persistence;
- (v) Appendix B: Admissible Representatives;
- (vi) Appendix D: Tower Diagnostics and Type IV Defect Calculus;
- (vii) Appendix M: Ledger Semantics and Integrated Defect Catalog;
- (viii) Chapter 7: Core Sovereignty and typed audit semantics;
- (ix) Chapter 8: Explicit Non-Claims and Core/interface boundary;
- (x) Appendix U: Agent Semantics;
- (xi) Appendix X: Validity Map and Global Certificate DAG;
- (xii) Companion 1: Arithmetic Calibration Companion.

Its role is to support disciplined execution, replay, and audit without converting operational reproducibility into proof.

Status labels for Companion 2. *Operational companion definitions:* Definitions [1.51](#), [1.52](#), [1.54](#), [1.56](#), [1.58](#), [1.60](#), [1.62](#), [1.64](#), [1.66](#), [1.68](#), [1.70](#), [1.71](#), [1.72](#), [1.75](#), [1.77](#), [1.45](#), [1.46](#), [1.47](#), [1.48](#).

Operational companion propositions: Propositions [1.49](#), [1.80](#), and [1.81](#).

Operational cautions: Remarks [1.42](#), [1.43](#), [1.44](#), [1.53](#), [1.55](#), [1.57](#), [1.59](#), [1.61](#), [1.63](#), [1.65](#), [1.67](#), [1.69](#), [1.73](#), [1.74](#), [1.76](#), [1.78](#), [1.79](#), [1.82](#), [1.50](#).

Explicit exclusions: Section C2.20.

Final companion closure: Proposition [1.81](#).